

LEARNING PLAN – APPLY PROGRAMMING PRINCIPLES

INSTITUTIONAL DETAILS

ITEM	DETAILS
Unit of Competence	Apply Programming Principles
Unit Code	0613 451 05A
Name of Trainer	JAMES NDIRANGU
Competence Level	LEVEL 5
Institution	THE ABERDARE TECHNICAL COLLEGE
Number of Learners	3
Date of Preparation	06/08/2026
Skill or Job Task	Applying computer programming skills, demonstrating structured programming skills, and demonstrating object-oriented programming skills.

BENCHMARK / CRITERIA TO BE USED

1. Apply computer programming skills

- *1.1 Programming language types are identified as per the user requirements.*
- *1.2 Programming paradigms are applied as per user requirements.*
- *1.3 Program development life cycle is applied as per the work requirements.*
- *1.4 Program design tools are applied as per the user requirements.*
- *1.5 Program writing tools are identified as per the system requirements.*

2. Demonstrate structured programming skills

- *2.1 Identifiers are declared as per program design specification.*
- *2.2 Initialization of variables and constants is performed according to program design specifications.*

- *2.3 User-defined data types are applied as per system requirements.*
- *2.4 Computer program input is created as per program design.*
- *2.5 Data control structures in a program are applied as per program design requirements.*
- *2.6 Data structures in a program are applied as per program design specifications.*
- *2.7 Computer program subroutines are created as per user needs.*
- *2.8 Computer program output is coded as per user requirements.*
- *2.9 Computer program debugging is performed as per work procedures.*
- *2.10 Computer program is compiled as per system requirements.*

3. Demonstrate object-oriented programming skills

- *3.1 Objects and classes are implemented as per work procedures.*
- *3.2 Objects methods are declared as per application requirements.*
- *3.3 Namespaces are applied as per work procedures.*
- *3.4 Data abstraction concepts are applied as per work procedures.*
- *3.5 Object encapsulations are applied as per work procedures.*
- *3.6 Class templates are implemented as per application requirements.*
- *3.7 Class inheritance is implemented as per application requirements.*
- *3.8 Polymorphism is implemented as per application requirements.*

LEARNING OUTCOME 1: APPLY COMPUTER PROGRAMMING SKILLS

WEEK 1

[illegible]

		• Functional languages	4. Differentiate programming language categories. 5. Classify given programming languages according to their categories.	their characteristics. 4. Compares language categories and asks trainees to differentiate them. 5. Provides programming language examples and guides trainees in classifying them.	programming language categories. 5. Classify the given programming languages according to their categories.	(11th ed.). Pearson.		
1	1. Apply computer programming skills	Session 2: Selection of Programming Languages • Criteria for selecting programming languages	By the end of the session, trainees should be able to: 1. Identify factors considered when selecting	1. Introduces language selection factors and poses questions on their importance. 2. Explains how user requirements	1. Identify factors considered when selecting programming languages and answer questions. 2. Explain how user	• Computers, projector, programming language comparison charts, case studies • Ref: Pratt, T. W., & Zelkowitz, M. V. (2001).	Language selection case study	Date: Reflection:

		• User requirements • System requirements • Project requirements • Developer considerations	programming languages. 2. Explain how user requirements influence language selection. 3. Analyse programming requirements. 4. Compare programming languages for a given project. 5. Select an appropriate programming language for a given requirement.	influence selection and asks trainees to identify examples. 3. Provides project requirements and guides trainees in analysing them. 4. Provides language alternatives and guides trainees in comparing them. 5. Presents project scenarios and asks trainees to select suitable languages.	requirements influence language selection. 3. Analyse the project requirements provided. 4. Compare the programming language alternatives. 5. Select the most appropriate programming language for each scenario.	<i>Programming Languages: Design and Implementation (4th ed.). Prentice Hall.</i>		
--	--	--	--	--	---	--	--	--

WEEK 2

Week	Main Learning Outcome	Session Title / Subtopics	Learning Outcome	Trainer Activities	Trainee Activities	Resources and References	Assessment	Reflections and Dates
2	1. Apply computer programming skills	Session 3: Application Programming Paradigms • Common programming paradigms • Functional • Procedural • Object-oriented • Imperative • Declarative	By the end of the session, trainees should be able to: 1. Define programming paradigms. 2. Identify common programming paradigms. 3. Explain the characteristics of each paradigm. 4. Differentiate imperative, declarative, procedural,	1. Introduces programming paradigms and poses questions about their purpose. 2. Identifies common paradigms and asks trainees to identify examples. 3. Explains the characteristics of each paradigm and asks trainees to describe them.	1. Define programming paradigms and answer questions about their purpose. 2. Identify common programming paradigms and their examples. 3. Explain the characteristics of each paradigm. 4. Differentiate the programming paradigms.	• Computers, projector, paradigm diagrams, sample programs • Ref: Van Roy, P., & Haridi, S. (2004). <i>Concepts, Techniques, and Models of Computer Programming</i>. MIT Press.	Paradigm comparison exercise, oral questions	Date: Reflection:

			functional, and object-oriented paradigms. 5. Analyse applications of different programming paradigms.	4. Compares paradigms and guides trainees in differentiating them. 5. Provides application scenarios and guides trainees in analysing suitable paradigms.	5. Analyse the application scenarios and identify suitable paradigms.			
2	1. Apply computer programming skills	Session 4: Selection of Programming Paradigms • Choosing appropriate paradigms • Project requirements	By the end of the session, trainees should be able to: 1. Identify factors influencing paradigm selection. 2. Explain the relationship	1. Explains factors affecting paradigm selection and poses questions about them. 2. Demonstrates how project needs influence paradigm	1. Identify factors influencing paradigm selection and answer questions. 2. Explain how project needs influence paradigm selection.	• Computers, case studies, projector, paradigm charts • Ref: Gabbrielli, M., & Martini, S. (2010). <i>Programming Languages: Principles and</i>	Paradigm selection case study	Date: Reflection:

		• User requirements • Project complexity	between project needs and paradigms. 3. Analyse project requirements. 4. Evaluate alternative programming paradigms. 5. Select an appropriate paradigm for a given project.	selection and asks trainees to identify relationships. 3. Provides project requirements and guides trainees in analysing them. 4. Provides alternative paradigms and guides trainees in evaluating them. 5. Provides project scenarios and asks trainees to select appropriate paradigms.	3. Analyse the project requirements provided. 4. Evaluate alternative programming paradigms. 5. Select an appropriate paradigm for the given project.	<i>Paradigms.</i> <i>Springer.</i>		
--	--	---	---	---	--	---	--	--

WEEK 3

Week	Main Learning Outcome	Session Title / Subtopics	Learning Outcome	Trainer Activities	Trainee Activities	Resources and References	Assessment	Reflections and Dates
3	1. Apply computer programming skills	Session 5: Program Development Life Cycle • Stages of program development life cycle • Requirements analysis • Program design • Coding • Testing • Implementation	By the end of the session, trainees should be able to: 1. Define program development life cycle. 2. Identify stages of the program development life cycle. 3. Explain the purpose of each stage. 4. Sequence the stages correctly.	1. Introduces the program development life cycle and poses questions about its purpose. 2. Identifies the stages and asks trainees to identify them. 3. Explains the purpose of each stage and asks trainees to describe it. 4. Demonstrates the correct	1. Define the program development life cycle and answer questions about its purpose. 2. Identify the stages of the life cycle. 3. Explain the purpose of each stage. 4. Sequence the stages correctly. 5. Analyse the importance	• Computers, projector, PDLC charts, notes • Ref: <i>Pressman, R. S. (2014). Software Engineering: A Practitioner's Approach (8th ed.). McGraw-Hill.</i>	Stage sequencing exercise, oral questions	Date: Reflection:

		• Maintenance	5. Analyse the importance of following the program development life cycle.	sequence and guides trainees in arranging the stages. 5. Provides development scenarios and guides trainees in analysing the importance of the life cycle.	of following the life cycle.			
3	1. Apply computer programming skills	Session 6: Adapting the Program Development Life Cycle • Best practices • Adapting the life cycle to work requirements	By the end of the session, trainees should be able to: 1. Identify best practices in program development. 2. Explain factors affecting	1. Introduces programming best practices and poses questions on their importance. 2. Explains factors affecting life-cycle adaptation and asks trainees to	1. Identify programming best practices and answer questions about their importance. 2. Explain factors affecting life-cycle adaptation.	• Computers, project scenarios, development templates • Ref: Somervell, I. (2015). <i>Software Engineering (10th ed.)</i>. Pearson.	Project development exercise	Date: Reflection:

		• Project constraints • User requirements	adaptation of the life cycle. 3. Analyse work requirements. 4. Evaluate alternative development approaches. 5. Develop an appropriate program development approach for a given project.	identify them. 3. Provides work requirements and guides trainees in analysing them. 4. Provides development approaches and guides trainees in evaluating them. 5. Provides a project scenario and guides trainees in developing an appropriate approach.	3. Analyse the work requirements provided. 4. Evaluate alternative development approaches. 5. Develop an appropriate development approach for the given project.			
--	--	--	---	--	---	--	--	--

WEEK 4

Week	Main Learning Outcome	Session Title / Subtopics	Learning Outcome	Trainer Activities	Trainee Activities	Resources and References	Assessment	Reflections and Dates
4	1. Apply computer programming skills	Session 7: Program Design Tools • Flowcharts • Decision tables • Decision trees • Pseudocode	By the end of the session, trainees should be able to: 1. Identify common program design tools. 2. Explain the purpose of flowcharts, decision tables, decision trees, pseudocode, and algorithms. 3. Differentiate program design tools.	1. Introduces program design tools and poses questions about their uses. 2. Explains each design tool and asks trainees to state its purpose. 3. Compares design tools and guides trainees in differentiating them. 4. Demonstrates algorithm and pseudocode	1. Identify program design tools and answer questions about their uses. 2. Explain the purpose of each design tool. 3. Differentiate the design tools. 4. Construct algorithms and pseudocode for given problems.	• Computers, projector, flowchart software, draw.io, worksheets	Algorithm, pseudocode, and flowchart practical. Projects Quizzes and tests	Date: Reflection:

		Algorithms	4. Construct algorithms and pseudocode. 5. Develop flowcharts for given programming problems.	development and guides trainees in constructing them. 5. Demonstrates flowchart development and guides trainees in developing flowcharts.	5. Develop flowcharts for the programming problems provided.	Ref: Cormen, T. H., et al. (2009). <i>Introduction to Algorithms (3rd ed.)</i>. MIT Press.		
4	1. Apply computer programming skills	Session 8: Selection of Program Design Tools - User requirements - Project complexity	By the end of the session, trainees should be able to: 1. Identify factors affecting selection of design tools. 2. Explain the relationship between	1. Explains factors affecting design-tool selection and poses questions about them. 2. Demonstrates how requirements influence tool selection and asks trainees	1. Identify factors affecting design-tool selection and answer questions. 2. Explain how requirements influence design-tool selection.	- Computers, case studies, draw.io, design worksheets - Ref: Robertson, S., & Robertson, J. (2012). <i>Mastering the Requirements Process (3rd ed.)</i>. Addison-Wesley.	Design-tool selection exercise	Date: Reflection:

		• Selecting appropriate design tools	requirements and design tools. 3. Analyse programming problems. 4. Evaluate alternative design tools. 5. Select appropriate design tools for given programming problems.	to identify examples. 3. Provides programming problems and guides trainees in analysing them. 4. Provides alternative tools and guides trainees in evaluating them. 5. Provides project scenarios and asks trainees to select suitable tools.	3. Analyse the programming problems provided. 4. Evaluate alternative design tools. 5. Select appropriate design tools for the given problems.			
--	--	--	--	---	---	--	--	--

WEEK 5

Week	Main Learning Outcome	Session Title / Subtopics	Learning Outcome	Trainer Activities	Trainee Activities	Resources and References	Assessment	Reflections and Dates
5	1. Apply computer programming skills	Session 9: Program Writing Tools and IDEs • Text editors • Compilers • Linkers • Debuggers • Integrated Development Environments	By the end of the session, trainees should be able to: 1. Identify common program writing tools. 2. Explain the functions of text editors, compilers, linkers, and debuggers. 3. Describe the functions of an IDE. 4. Differentiate	1. Introduces program writing tools and poses questions about their purposes. 2. Explains text editors, compilers, linkers, and debuggers and asks trainees to identify their functions. 3. Demonstrates IDE features and asks trainees to identify them.	1. Identify program writing tools and answer questions about their purposes. 2. Explain the functions of text editors, compilers, linkers, and debuggers. 3. Identify the main features of an IDE. 4. Differentiate program writing tools according to their functions.	• Computers, C/C++ compiler, IDE, text editor, projector • Ref: Aho, A. V., et al. (2006). <i>Compilers: Principles, Techniques, and Tools</i> (2nd ed.). Addison-Wesley.	Tool identification, practical exercise Practical tests	Date: Reflection:

			program writing tools. 5. Demonstrate the use of basic program writing tools.	4. Compares program writing tools and guides trainees in differentiating them. 5. Demonstrates basic tool usage and guides trainees in practising it.	5. Demonstrate basic use of the program writing tools.			
5	1. Apply computer programming skills	Session 10: Evaluation and Selection of Programming Tools • System requirements • Developer preferences	By the end of the session, trainees should be able to: 1. Identify factors used to evaluate programming tools. 2. Explain how system	1. Explains tool evaluation criteria and poses questions on their importance. 2. Demonstrates the relationship between system	1. Identify programming-tool evaluation criteria and answer questions. 2. Explain how system requirements influence tool selection.	• Computers, IDEs, compiler, linker, debugger • <i>Ref: Kernighan, B. W., & Pike, R. (1999). The Practice of Programming. Addison-Wesley.</i>	CAT 1: Programming Languages, Paradigms,	Date: Reflection:

		• Tool compatibility • IDE selection	requirements influence tool selection. 3. Analyse developer and project requirements. 4. Evaluate available programming tools. 5. Select appropriate programming tools for a project.	requirements and tools and asks trainees to identify suitable examples. 3. Provides project requirements and guides trainees in analysing them. 4. Provides different programming tools and guides trainees in evaluating them. 5. Provides project scenarios and asks trainees to select suitable tools.	3. Analyse the project and developer requirements. 4. Evaluate the available programming tools. 5. Select suitable programming tools for the project.		PDLC and Design Tools	
--	--	---	---	---	--	--	------------------------------	--

--	--	--	--	--	--	--	--	--

LEARNING OUTCOME 2: DEMONSTRATE STRUCTURED PROGRAMMING SKILLS

WEEK 6

Week	Main Learning Outcome	Session Title / Subtopics	Learning Outcome	Trainer Activities	Trainee Activities	Resources and References	Assessment	Reflections and Dates
6	2. Demonstrate structured programming skills	Session 11: C Identifiers, Variables and Constants - Declaration of identifiers - Naming conventions - Best practices - Variables	By the end of the session, trainees should be able to: - Define identifiers, variables, and constants. - Identify valid and invalid C identifiers. - Apply naming conventions when	- Introduces C identifiers, variables, and constants and poses questions about their purposes. - Demonstrates valid and invalid identifiers and asks trainees to identify them. - Explains naming conventions	- Define identifiers, variables, and constants and answer questions about their purposes. - Identify valid and invalid C identifiers. - Apply correct naming conventions when declaring identifiers.	- Computers, C compiler/IDE, projector, C programming notes - Ref: Kernighan, B. W., & Ritchie, D. M. (1988). <i>The C Programming Language</i> (2nd ed.). Prentice Hall.	Identifier and initialization practical. Projects	Date: Reflection:

		• Constants • Initialization	declaring identifiers. 4. Initialize variables and constants correctly. 5. Evaluate declarations against program design specifications.	and guides trainees in applying them. 4. Demonstrates variable and constant initialization and guides trainees in practising it. 5. Provides declarations and asks trainees to evaluate them against specifications.	4. Initialize variables and constants correctly. 5. Evaluate declarations against the given specifications.		Quizzes and tests	
6	2. Demonstrate structured programming skills	Session 12: User-Defined Data Types in C • Structures	By the end of the session, trainees should be able to: 1. Identify common data	1. Introduces data types and poses questions about their uses.	1. Identify common C data types and answer questions about their uses.	• Computers, C compiler/IDE, sample programs • <i>Ref: King, K. N. (2008). C</i>		Date: Reflection:

		• Classes • Arrays • Functions • Criteria for selecting data types	types used in C programming. 2. Explain structures, arrays, and functions. 3. Differentiate the appropriate uses of data types. 4. Apply suitable data types to programming problems. 5. Select appropriate data types based on system requirements.	2. Explains structures, arrays, and functions and asks trainees to describe them. 3. Compares data types and guides trainees in differentiating their uses. 4. Demonstrates data-type application and guides trainees in applying suitable types. 5. Provides system requirements and asks trainees to select	2. Explain structures, arrays, and functions. 3. Differentiate the uses of different data types. 4. Apply suitable data types to programming problems. 5. Select appropriate data types based on the given requirements.	<i>Programming: A Modern Approach (2nd ed.). W. W. Norton & Company.</i>	Data-type selection practical	
--	--	---	---	---	--	---	-------------------------------	--

				appropriate types.				
--	--	--	--	--------------------	--	--	--	--

WEEK 7

Week	Main Learning Outcome	Session Title / Subtopics	Learning Outcome	Trainer Activities	Trainee Activities	Resources and References	Assessment	Reflections and Dates
7	2. Demonstrate structured programming skills	Session 13: Creating C Program Input and Output • Program input • Input statements • Program output • Output statements	By the end of the session, trainees should be able to: 1. Explain program input and output. 2. Identify appropriate input and output statements. 3. Apply input statements in C programs.	1. Explains program input and output and poses questions about their purposes. 2. Demonstrates input and output statements and asks trainees to identify them. 3. Demonstrates input coding and guides trainees in	1. Explain program input and output and answer questions. 2. Identify appropriate input and output statements. 3. Apply input statements in C programs. 4. Apply output statements in C programs.	• Computers, C compiler/IDE, sample programs • <i>Ref: Prata, S. (2013). C Primer Plus (6th ed.). Addison-Wesley.</i>	Input/output programming practical. Projects	Date: Reflection:

			4. Apply output statements in C programs. 5. Create C programs that accept input and produce output.	applying input statements. 4. Demonstrates output coding and guides trainees in applying output statements. 5. Provides programming tasks and guides trainees in creating complete input/output programs.	5. Create C programs that accept input and produce the required output.		Quizzes and tests	
7	2. Demonstrate structured programming skills	Session 14: C Control Structures • Sequence	By the end of the session, trainees should be able to: 1. Identify types of	1. Introduces control structures and poses identification questions.	1. Identify control structures and answer identification questions.	• Computers, C compiler/IDE, flowcharts, sample programs	Control-structure practical	Date: Reflection:

		• Selection • Loops • Best practices in control structures	control structures. 2. Explain sequence, selection, and iteration. 3. Differentiate the control structures. 4. Apply appropriate control structures in C programs. 5. Evaluate control structures against design requirements.	2. Explains sequence, selection, and loops and asks trainees to describe them. 3. Compares control structures and guides trainees in differentiating them. 4. Demonstrates control structures and guides trainees in applying them. 5. Provides program designs and asks trainees to evaluate the selected	2. Explain sequence, selection, and loops. 3. Differentiate the control structures. 4. Apply appropriate control structures in C programs. 5. Evaluate the control structures against the design requirements.	• <i>Ref: Deitel, P., & Deitel, H. (2015). C How to Program (8th ed.). Pearson.</i>		
--	--	--	---	--	--	--	--	--

				control structures.				
--	--	--	--	---------------------	--	--	--	--

WEEK 8

Week	Main Learning Outcome	Session Title / Subtopics	Learning Outcome	Trainer Activities	Trainee Activities	Resources and References	Assessment	Reflections and Dates
8	2. Demonstrate structured programming skills	Session 15: Data Structures in C • Arrays • Queues • Stacks • Linked lists • Selection of appropriate data structures	By the end of the session, trainees should be able to: 1. Identify common data structures. 2. Explain arrays, queues, stacks, and linked lists. 3. Differentiate the data structures.	1. Introduces data structures and poses questions about their purpose. 2. Explains arrays, queues, stacks, and linked lists and asks trainees to describe them. 3. Compares data structures and guides trainees in differentiating them. 4. Demonstrates	1. Identify common data structures and answer questions about their purpose. 2. Explain arrays, queues, stacks, and linked lists. 3. Differentiate the data structures. 4. Apply data structures in C programs.	• Computers, C compiler/IDE, data structure diagrams, sample programs • Ref: <i>Forouzan, B. A., & Gilberg, R. F. (2006). Computer Science: A Structured Programming Approach Using C (3rd ed.). Cengage.</i>	Data structure practical. Projects Quizzes and tests	Date: Reflection:

			4. Apply data structures in C programs. 5. Select appropriate data structures based on design specifications.	data structure implementation and guides trainees in applying them. 5. Provides design specifications and asks trainees to select suitable structures.	5. Select appropriate data structures for the given specifications.			
8	2. Demonstrate structured programming skills	Session 16: C Program Subroutines and Functions • Benefits of subroutines • Functions and subprograms • Designing subroutines	By the end of the session, trainees should be able to: 1. Explain the purpose of subroutines. 2. Identify benefits of using subroutines.	1. Explains subroutines and poses questions about their benefits. 2. Demonstrates benefits of modular programming and asks trainees to identify them.	1. Explain the purpose of subroutines and answer questions about their benefits. 2. Identify benefits of using subroutines. 3. Differentiate	• Computers, C compiler/IDE, sample programs • <i>Ref: Hanly, J. R., & Koffman, E. B. (2015). Problem Solving and Program Design in C (8th ed.). Pearson.</i>	Functions/subroutines practical	Date: Reflection:

		• User needs	3. Differentiate functions and subprograms. 4. Design subroutines to meet user needs. 5. Create C functions and subroutines.	3. Explains functions and subprograms and guides trainees in differentiating them. 4. Demonstrates subroutine design and guides trainees in designing subroutines. 5. Demonstrates function creation and guides trainees in creating functions.	functions and subprograms. 4. Design subroutines to meet the stated user needs. 5. Create C functions and subroutines.			
--	--	--	---	--	---	--	--	--

WEEK 9

Week	Main Learning Outcome	Session Title / Subtopics	Learning Outcome	Trainer Activities	Trainee Activities	Resources and References	Assessment	Reflections and Dates
9	2. Demonstrate structured programming skills	Session 17: C Program Debugging • Common debugging techniques • Debugging tools • Systematic debugging procedures	By the end of the session, trainees should be able to: 1. Define program debugging. 2. Identify common programming errors. 3. Explain common debugging techniques. 4. Apply systematic debugging procedures.	1. Explains debugging and poses questions about its importance. 2. Demonstrates common programming errors and asks trainees to identify them. 3. Demonstrates debugging techniques and guides trainees in applying them.	1. Define debugging and answer questions about its importance. 2. Identify errors in the sample programs. 3. Explain and apply common debugging techniques. 4. Follow systematic debugging procedures. 5. Debug the faulty C	• Computers, C compiler/IDE, debugger, faulty programs • Ref: Agans, D. J. (2002). <i>Debugging: The 9 Indispensable Rules for Finding Even the Most Elusive Software and Hardware Problems.</i> AMACOM.	Debugging practical	Date:

			5. Debug C programs using appropriate tools.	4. Demonstrates systematic debugging procedures and guides trainees through the process. 5. Provides faulty C programs and guides trainees in debugging them.	programs using appropriate tools.			Reflection:
9	2. Demonstrate structured programming skills	Session 18: C Program Compilation • Compilation process • Compiler • Linker	By the end of the session, trainees should be able to: 1. Explain the compilation process.	1. Explains the compilation process and poses questions about its stages. 2. Demonstrates the compilation	1. Explain the compilation process and answer questions about its stages. 2. Identify the stages involved in compilation.	• Computers, C compiler, IDE, sample programs • <i>Ref: Levine, J. R. (2000). Linkers and Loaders. Morgan Kaufmann.</i>	Compilation practical	Date: Reflection:

		• Executable program • System requirements	2. Identify the stages involved in compilation. 3. Describe the roles of the compiler and linker. 4. Compile C programs successfully. 5. Evaluate compiled programs against system requirements.	stages and asks trainees to identify them. 3. Explains compiler and linker functions and asks trainees to describe them. 4. Demonstrates C program compilation and guides trainees in compiling programs. 5. Provides system requirements and asks trainees to evaluate their compiled programs.	3. Describe the roles of the compiler and linker. 4. Compile C programs successfully. 5. Evaluate the compiled programs against the given requirements.				
--	--	---	--	--	--	--	--	--	--

WEEK 10

[illegible]

		• Functions • Debugging • Compilation	data structures. 5. Debug the developed program. 6. Compile and evaluate the completed C program.	4. Demonstrates function and data structure integration and guides trainees in creating them. 5. Provides debugging support and guides trainees in correcting errors. 6. Supervises compilation and asks trainees to evaluate their completed programs.	5. Debug the developed program and correct errors. 6. Compile and evaluate the completed C program.		Quizzes and tests	
--	--	---	--	--	---	--	-------------------	--

LEARNING OUTCOME 3: DEMONSTRATE OBJECT-ORIENTED PROGRAMMING SKILLS

WEEK 11

Week	Main Learning Outcome	Session Title / Subtopics	Learning Outcome	Trainer Activities	Trainee Activities	Resources and References	Assessment	Reflections and Dates
11	3. Demonstrate object-oriented programming skills	Session 20: Objects and Classes in C++ • Introduction to OOP • Objects • Classes • Work procedures	By the end of the session, trainees should be able to: 1. Define object-oriented programming. 2. Identify objects and classes. 3. Explain the relationship between objects and classes.	1. Introduces OOP and poses questions about its purpose. 2. Explains objects and classes and asks trainees to identify them. 3. Demonstrates the relationship between classes and objects and asks trainees to explain it.	1. Define OOP and answer questions about its purpose. 2. Identify objects and classes from the given examples. 3. Explain the relationship between objects and classes. 4. Differentiate	• Computers, C++ IDE/compiler, projector, OOP notes • Ref: Stroustrup, B. (2013). <i>The C++ Programming Language (4th ed.)</i>. Addison-Wesley.	Class and object	Date: Reflection:

			4. Differentiate objects from classes. 5. Create basic C++ classes and objects.	4. Compares objects and classes and guides trainees in differentiating them. 5. Demonstrates class and object creation and guides trainees in creating them.	objects from classes. 5. Create basic C++ classes and objects.		creation practical	
11	3. Demonstrate object-oriented programming skills	Session 21: Object Methods in C++ • Object methods • Method declaration	By the end of the session, trainees should be able to: 1. Define object methods.	1. Introduces object methods and poses questions about their purpose. 2. Explains method functionality and asks	1. Define object methods and answer questions about their purpose. 2. Explain method functionality	• Computers, C++ IDE/compiler, sample programs • <i>Ref: Lippman, S. B., Lajoie, J., & Moo, B. E. (2012). C++ Primer (5th</i>	Method creation practical. Projects	Date: Reflection:

		• Application requirements • Method naming and functionality	2. Explain the purpose of methods. 3. Declare object methods in C++. 4. Apply appropriate method naming conventions. 5. Create methods that fulfil application requirements.	trainees to identify suitable uses. 3. Demonstrates method declaration and guides trainees in declaring methods. 4. Demonstrates method naming practices and asks trainees to apply them. 5. Provides application requirements and guides trainees in creating suitable methods.	and identify suitable uses. 3. Declare object methods in C++. 4. Apply appropriate method naming practices. 5. Create methods that fulfil the given application requirements.	<i>ed.). Addison-Wesley.</i>	Quizzes and tests	
--	--	---	--	--	---	-------------------------------------	-------------------	--

WEEK 12

Week	Main Learning Outcome	Session Title / Subtopics	Learning Outcome	Trainer Activities	Trainee Activities	Resources and References	Assessment	Reflections and Dates
12	3. Demonstrate object-oriented programming skills	Session 22: Namespaces and Data Abstraction in C++ • Role of namespaces • Implementing namespaces • Definition of data abstraction • Importance of data abstraction • Implementing	By the end of the session, trainees should be able to: 1. Explain the purpose of namespaces. 2. Implement namespaces in C++ programs. 3. Define data abstraction. 4. Explain the importance of data abstraction.	1. Explains namespaces and poses questions about their purpose. 2. Demonstrates namespace implementation and guides trainees in implementing namespaces. 3. Explains data abstraction and asks trainees to define it. 4. Discusses the importance of abstraction and asks trainees to	1. Explain the purpose of namespaces and answer questions about their use. 2. Implement namespaces in C++ programs. 3. Define data abstraction. 4. Explain the importance and benefits of data abstraction.	• Computers, C++ IDE/compiler, OOP diagrams, sample programs • <i>Ref: Lafore, R. (2002). Object-Oriented Programming in C++ (4th ed.). Sams Publishing.</i>	Namespace and abstraction practical. Projects	Date:

		data abstraction	5. Implement data abstraction in C++ programs.	explain its benefits. 5. Demonstrates abstraction implementation and guides trainees in implementing it.	5. Implement data abstraction in C++ programs.		Quizzes and tests	Reflection:
12	3. Demonstrate object-oriented programming skills	Session 23: Object Encapsulation in C++ • Definition of encapsulation • Importance of encapsulation • Implementing encapsulation	By the end of the session, trainees should be able to: 1. Define object encapsulation. 2. Explain the importance of encapsulation. 3. Identify encapsulation	1. Defines encapsulation and poses questions about its meaning. 2. Explains the importance of encapsulation and asks trainees to identify its benefits. 3. Demonstrates encapsulated classes and	1. Define encapsulation and answer questions about its meaning. 2. Explain the importance of encapsulation and identify its benefits. 3. Identify encapsulation in sample	• Computers, C++ IDE/compiler, sample programs • <i>Ref: Deitel, P., & Deitel, H. (2017). C++ How to Program (10th ed.). Pearson.</i>	Encapsulation practical	Date:

			in C++ programs. 4. Apply access control mechanisms. 5. Implement encapsulation in C++ classes.	asks trainees to identify encapsulation. 4. Demonstrates access control and guides trainees in applying it. 5. Demonstrates encapsulation implementation and guides trainees in implementing it.	C++ programs. 4. Apply appropriate access control mechanisms. 5. Implement encapsulation in C++ classes.			Reflection:
--	--	--	--	---	---	--	--	---------------------------

[illegible]

			C++ programs. 5. Create a C++ program using class templates.	trainees in applying templates. 5. Provides a programming task and guides trainees in creating a template-based program.	5. Create a C++ program using a class template.	• <i>Ref: Vandevoorde, D., Josuttis, N. M., & Gregor, D. (2017). C++ Templates: The Complete Guide (2nd ed.). Addison-Wesley.</i>		
13	3. Demonstrate object-oriented programming skills	Session 25: Class Inheritance in C++ • Base classes • Derived classes • Inheritance relationships • Types of inheritance	By the end of the session, trainees should be able to: 1. Define inheritance. 2. Explain the importance of inheritance.	1. Introduces inheritance and poses questions about its purpose. 2. Explains the importance of inheritance and asks trainees to explain its benefits. 3. Demonstrates base and derived classes and asks	1. Define inheritance and answer questions about its purpose. 2. Explain the importance of inheritance and state its benefits. 3. Identify base and derived	• Computers, C++ IDE/compiler, inheritance diagrams, sample programs • <i>Ref: Meyers, S. (2005). Effective C++: 55 Specific Ways to Improve Your Programs and Designs (3rd ed.).</i>	Inheritance practical	Date: Reflection:

			3. Identify base and derived classes. 4. Differentiate types of inheritance. 5. Implement inheritance relationships in C++.	trainees to identify them. 4. Compares types of inheritance and guides trainees in differentiating them. 5. Demonstrates inheritance implementation and guides trainees in implementing relationships.	classes in the examples provided. 4. Differentiate the types of inheritance. 5. Implement inheritance relationships in C++.	<i>Addison-Wesley.</i> Curriculum Occupational standards		
--	--	--	--	---	--	---	--	--

WEEK 14

Week	Main Learning Outcome	Session Title / Subtopics	Learning Outcome	Trainer Activities	Trainee Activities	Resources and References	Assessment	Reflections and Dates
14	3. Demonstrate object-oriented programming skills	Session 26: Polymorphism in C++ • Definition of polymorphism • Importance of polymorphism • Implementing polymorphism	By the end of the session, trainees should be able to: 1. Define polymorphism. 2. Explain the importance of polymorphism. 3. Identify polymorphism in C++ programs. 4. Differentiate forms of polymorphism. 5. Implement polymorphism	1. Defines polymorphism and poses questions about its meaning. 2. Explains the importance of polymorphism and asks trainees to state its benefits. 3. Demonstrates polymorphic programs and asks trainees to identify polymorphism. 4. Compares forms of polymorphism and guides trainees in	1. Define polymorphism and answer questions about its meaning. 2. Explain the importance of polymorphism and state its benefits. 3. Identify polymorphism in the sample C++ programs. 4. Differentiate the forms of polymorphism. 5. Implement polymorphism	• Computers, C++ IDE/compiler, sample programs, OOP diagrams • Ref: Schildt, H. (2014). <i>C++: The Complete Reference (5th ed.)</i>. McGraw-Hill.	Polymorphism practical	Date: Reflection:

			in C++ programs.	differentiating them. 5. Demonstrates polymorphism implementation and guides trainees in implementing it.	in C++ programs.			
14	3. Demonstrate object-oriented programming skills	Session 27: Integrated Object-Oriented C++ Programming • Classes and objects • Methods • Namespaces • Abstraction	By the end of the session, trainees should be able to: 1. Analyse an object-oriented programming problem. 2. Design appropriate classes and objects. 3. Apply methods,	1. Provides an OOP programming problem and guides trainees in analysing it. 2. Demonstrates class and object design and guides trainees in designing classes. 3. Guides trainees in	1. Analyse the OOP programming problem provided. 2. Design appropriate classes and objects. 3. Apply methods, namespaces, abstraction, and encapsulation.	• Computers, C++ IDE/compiler, project specifications, sample programs	CAT 3 + Practical Assessment 2: Object-Oriented Programming	Date: Reflection:

		• Encapsulation • Class templates • Inheritance • Polymorphism	namespaces, abstraction, and encapsulation. 4. Implement class templates and inheritance. 5. Implement polymorphism. 6. Evaluate the completed C++ application against user requirements.	applying methods, namespaces, abstraction, and encapsulation. 4. Demonstrates templates and inheritance and guides trainees in implementing them. 5. Demonstrates polymorphism and guides trainees in implementing it. 6. Reviews completed applications and asks trainees to evaluate them	4. Implement class templates and inheritance. 5. Implement polymorphism in the application. 6. Evaluate the completed C++ application against user requirements.	• Ref: Stroustrup, B. (2013). <i>The C++ Programming Language (4th ed.)</i>. Addison-Wesley.		
--	--	---	---	---	---	--	--	--

				against requirements.				
--	--	--	--	--------------------------	--	--	--	--